

Heartbeat and Silent-Failure Detection

Learning goal. Understand how a quiet system detects a dead coordinator, why every replica owns a local heartbeat FSM, and how stale watchdog messages are prevented from starting false elections.

Implementation base: `origin/main` at 84846dc.

Election handoff inspected: `feature/electionTransaction` at d1222a2.

Key files: `HeartbeatTransaction.java`, `Replica.java`, heartbeat diagrams and tests.

1. The failure that application timeouts cannot see

If clients are actively writing, missing UPDATE or WRITEOK can reveal coordinator failure. But what if nobody sends a write for ten minutes? A dead coordinator would remain unnoticed.

Heartbeat solves this silent-period problem. The coordinator periodically broadcasts “I am alive.” Followers restart a watchdog whenever they receive a valid heartbeat. If a follower’s current watchdog expires, it asks the replica to start an election.

Heartbeat does not replace UPDATE or WRITEOK timeouts. It detects absence of general coordinator liveness, while those timers detect failure at specific update stages.

2. One local FSM per replica

After `InitSystem`, every replica constructs a local `HeartbeatTransaction`. These are separate Java objects with separate state and timers:

- coordinator’s local FSM sends heartbeats;
- each follower’s local FSM watches them.

They all use the same coordinator-scoped `TransactionId`:

```
<coordinator ActorRef, 0>
```

The shared ID is a correlation key, not a shared object. A heartbeat sent by the coordinator routes to the follower’s own local heartbeat FSM because both represent the same coordinator term.

3. Message vocabulary

`HeartbeatTickMsg`

A local scheduler message. It tells the coordinator FSM to emit the next heartbeat. It never crosses `NetworkChannel`.

`HeartbeatMsg`

A network message broadcast from coordinator to followers. It includes the expected numeric `coordinatorId`.

`WatchdogExpiredMsg`

A local scheduler message on a follower. It includes `watchdogVersion` so an old timeout can be identified.

This split makes the system easy to reason about: only `HeartbeatMsg` is remote evidence; tick and expiry are local clocks.

4. FSM states

```
STOPPED -> COORDINATOR  
STOPPED -> WATCHING -> ELECTION_REQUESTED
```

- **STOPPED**: constructed, not started.
- **COORDINATOR**: schedules ticks and broadcasts.
- **WATCHING**: follower maintains watchdog.
- **ELECTION_REQUESTED**: a current watchdog expired; do not request repeatedly.

Repeated `start()` calls outside **STOPPED** are ignored, preventing duplicate periodic timer chains.

5. Coordinator trace

- 1 `HeartbeatTransaction.start` compares owner replica ID with `coordinatorID`.
- 2 If equal, state becomes **COORDINATOR**.
- 3 It schedules one `HeartbeatTickMsg` after the configured beat interval.
- 4 When the tick arrives, replica routing returns it to this FSM.
- 5 The FSM verifies it is still in **COORDINATOR**.
- 6 It broadcasts `HeartbeatMsg` to every other replica through delayed FIFO channels.
- 7 It schedules the next one-shot tick.

Using repeated one-shot scheduling avoids maintaining a second periodic callback mechanism. Each iteration returns through the actor mailbox.

6. Follower trace

- 1 Start sees that owner is not coordinator.
- 2 State becomes **WATCHING**.
- 3 `restartWatchdog` increments the version and schedules expiry.
- 4 A valid heartbeat with the currently expected coordinator ID arrives.
- 5 The follower cancels the old timer if possible.
- 6 It increments version and schedules a new watchdog.

Heartbeats received in any other state or naming another coordinator do not reset the watchdog.

7. Watchdog budget

The current formula is:

```
3 * coordinatorBeatInterval + maxLatencyPlusTolerance
```

The intention is to tolerate several missed opportunities plus one delayed delivery/scheduling allowance. With a 1000 ms beat interval, the follower does not elect after a single heartbeat is late.

The formula is an implementation assumption. To defend it in an exam, relate each term to an actual path and configured network bound. Do not say merely “three seemed safe.”

8. The stale-timeout race

This is the most important educational detail.

```
watchdog version 4 scheduled
heartbeat arrives
version 4 timer cancelled
version 5 scheduled
version 4 expiry was already placed in mailbox
```

If the FSM reacted only to the message class, version 4 could start an election even though a heartbeat just arrived. It therefore compares `expired.watchdogVersion` with the current field. Mismatch means stale; ignore it.

This pattern is transferable to retries, leases, election attempts, and request generations.

9. Election handoff

When the current version expires in `WATCHING`:

- 1 state becomes `ELECTION_REQUESTED` first;
- 2 FSM calls `Replica.startElection(currentCoordinatorID)`.

Changing state first prevents a second queued expiry from triggering another request. `Replica.startElection` adds further deduplication and a deterministic delayed start based on ring distance.

On `origin/main`, this handoff was a `TODO`. The current election branch replaces it with the concrete call, so report provenance matters.

10. Restart after a new coordinator

Synchronization updates `coordinatorID` and epoch, removes the old heartbeat transaction from active routing, constructs a new local FSM with the new coordinator's `ActorRef` and sequence zero, and starts the correct role.

Old queued tick/expiry messages carry the old coordinator-scoped transaction ID. Because the old FSM is no longer active, they are dropped by dispatch. A new coordinator `ActorRef` also changes the ID.

11. Failure and validation boundaries

The FSM checks numeric coordinator ID but the inspected `handleHeartbeat` does not independently compare `heartbeat.sender` with the membership `ActorRef` for that coordinator. In the trusted course model, messages are not Byzantine, but sender validation still makes stale/malformed input behavior clearer.

A follower already in `ELECTION_REQUESTED` ignores heartbeats. Recovery must install a new heartbeat transaction rather than trying to revive the old term.

12. Tests and missing cases

Current regression tests show:

- coordinator broadcasts periodic heartbeat messages; and
- a scheduled tick routes to the heartbeat transaction by ID.

Important missing focused tests include:

- follower watchdog expiry starts exactly one election;
- a valid heartbeat postpones expiry;
- stale watchdog versions are ignored;
- wrong-coordinator and wrong-sender heartbeats do not reset;
- every replica initializes one local FSM with the shared ID; and
- heartbeat restarts correctly after synchronization.

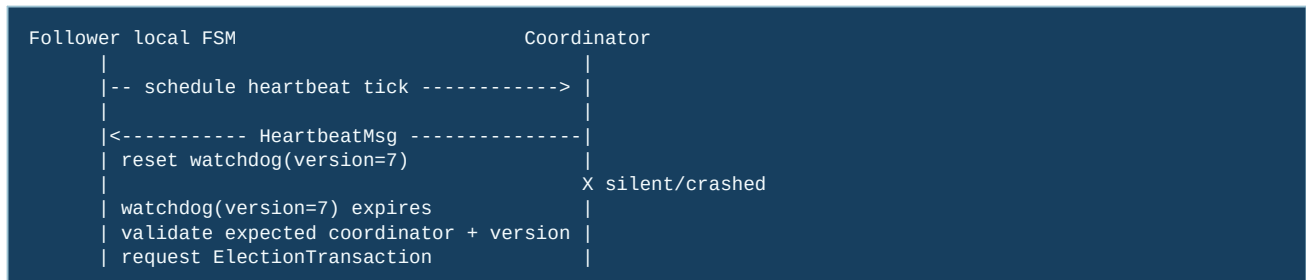
13. Exam rehearsal

Why not create one shared HeartbeatTransaction object? Actor state cannot be shared mutably. Each replica needs its own role and timer, while a shared ID correlates their messages.

Why is cancel plus version needed? Cancel may fail to remove an event already queued. Version validation decides whether the event is still current at handling time.

The invariant to remember is: **only a current follower watchdog for the currently expected coordinator may trigger one election request.**

14. Heartbeat as a local FSM plus a distributed signal



There is one `HeartbeatTransaction` object per replica, not one globally shared object. The coordinator instance sends heartbeats; follower instances maintain watchdogs. They share a coordinator-scoped `TransactionId` so routing can identify the term, while their timers and state remain actor-local.

15. Code microscope: why the version field exists

Consider this event order: watchdog v6 is scheduled; a heartbeat arrives and installs v7; cancellation of v6 races with the scheduler; v6 is delivered to the mailbox. A boolean “watchdog exists” is insufficient because v6 and v7 are both real queued events. The handler must compare the event’s version with the current field and ignore v6.

```
onHeartbeat(msg):
    if (msg.sender() != expectedCoordinator) return;
    watchdogVersion++;
    cancel(oldTimeout);
    timeout = schedule(WatchdogExpired(watchdogVersion));

onWatchdogExpired(expired):
    if (expired.version() != watchdogVersion) return; // stale event
    requestElectionOnce();
```

This is a general asynchronous-programming pattern: cancellation is an optimization; validation at consumption time is the correctness mechanism.

16. Timing and false suspicions

With heartbeat period H and channel delay bound L , a timeout near H can fire during a healthy delayed delivery. Add scheduler jitter and processing time when choosing the safety margin. Conversely, an enormous timeout delays election after a genuine crash. Explain the chosen bound in terms of the configuration, then test it with deterministic delay values rather than relying on one lucky random run.

Practice

Write a test where a wrong sender sends a heartbeat, then the real coordinator sends one. The wrong message must not reset the timer. Add a second test that delivers an old watchdog version after a fresh heartbeat and proves no duplicate election request is emitted.